



POLITECNICO
MILANO 1863

Prova finale di Reti Logiche
AA 2023-2024

Lorenzo Della Matera
Codice Persona: 10799739 - Matricola: 981843

Data: 28 marzo 2024

Indice

1	Introduzione	2
1.1	Specifica del progetto	2
1.2	Interfaccia del componente	3
2	Architettura	5
2.1	Schema del componente	5
2.2	Scelte progettuali	6
2.3	Moduli	6
2.3.1	counter_add	6
2.3.2	register	6
2.3.3	counter_c	7
2.3.4	counter_k	7
2.3.5	MUX	7
2.4	Segnali	7
2.5	FSM	8
2.5.1	Funzionamento	8
3	Risultati sperimentali	11
3.1	Report Utilization	11
3.2	Report Timing	11
3.3	Testing	12
4	Conclusioni	16

1 Introduzione

1.1 Specifica del progetto

Lo scopo del progetto è la realizzazione di un componente hardware che si interfacci con una memoria. Il modulo deve leggere un messaggio composto da K parole W da 8 bit memorizzate nella memoria, a partire dall'indirizzo ADD, ogni 2 byte. Il byte mancante deve essere riempito con un valore di credibilità C. Se la parola letta è diversa da zero il valore di credibilità è pari a 31. Altrimenti viene copiato l'ultimo valore letto e il valore di credibilità è decrementato.

Per esempio se all'indirizzo ADD la parola W è diversa da zero, allora all'indirizzo ADD+1 viene inserito il valore 31. Se poi all'indirizzo ADD+2 è presente una parola W pari a zero, C viene decrementato e all'indirizzo ADD+3 viene inserito il valore 30.

L'esecuzione termina quando sono state elaborate tutte le K parole.

Note:

- Il valore di C non può mai scendere sotto 0 e viene reinizializzato a 31 ogni volta che si incontra una parola diversa da zero. Una volta che C raggiunge il valore 0 non viene ulteriormente decrementato.
- Se il primo valore della sequenza è pari a zero, il suo valore non viene modificato e il valore di credibilità è posto a 0.

Esempio:

ADD = 1234 (0x4D2) K = 14

4D2	4D3	4D4	4D5	4D6	4D7	4D8	4D9	4DA	4DB	4DC	4DD	4DE	4DF
128	0	64	0	0	0	0	0	0	0	0	0	0	0
4E0	4E1	4E2	4E3	4E4	4E5	4E6	4E7	4E8	4E9	4EA	4EB	4EC	4ED
100	0	1	0	0	0	5	0	23	0	200	0	0	0

Tabella 1: Sequenza di partenza presente in memoria

4D2	4D3	4D4	4D5	4D6	4D7	4D8	4D9	4DA	4DB	4DC	4DD	4DE	4DF
128	31	64	31	64	30	64	29	64	28	64	27	64	26
4E0	4E1	4E2	4E3	4E4	4E5	4E6	4E7	4E8	4E9	4EA	4EB	4EC	4ED
100	31	1	31	1	30	5	31	23	31	200	31	200	30

Tabella 2: Sequenza finale in memoria dopo l'elaborazione

1.2 Interfaccia del componente

Il componente da implementare ha la seguente interfaccia:

```
entity project_reti_logiche is
  port (
    i_clk    : in std_logic;
    i_rst    : in std_logic;
    i_start  : in std_logic;
    i_add    : in std_logic_vector(15 downto 0);
    i_k      : in std_logic_vector(9 downto 0);

    o_done   : out std_logic;

    o_mem_addr : out std_logic_vector(15 downto 0);
    i_mem_data : in std_logic_vector(7 downto 0);
    o_mem_data : out std_logic_vector(7 downto 0);
    o_mem_we   : out std_logic;
    o_mem_en   : out std_logic
  );
end project_reti_logiche;
```

Figura 1: Entità principale

Segnali di input

- `i_clk` è il segnale di CLOCK generato dal Test Bench;
- `i_start` è il segnale di START generato dal Test Bench;
- `i_k` è il segnale (vettore) W generato dal Test Bench rappresentante la lunghezza della sequenza;
- `i_add` è il segnale (vettore) ADD generato dal Test Bench che rappresenta l'indirizzo;
- `i_mem_data` è il segnale (vettore) che arriva dalla memoria e contiene il dato in seguito ad una richiesta di lettura dal quale parte la sequenza da elaborare.

Segnali di output

- `o_done` è il segnale DONE di uscita che comunica la fine dell'elaborazione;
- `o_mem_addr` è il segnale (vettore) di uscita che manda l'indirizzo alla memoria;
- `o_mem_data` è il segnale (vettore) che va verso la memoria e contiene il dato che verrà successivamente scritto;
- `o_mem_en` è il segnale di ENABLE da dover mandare alla memoria per poter comunicare (sia in lettura che in scrittura);
- `o_mem_we` è il segnale di WRITE ENABLE da dover mandare alla memoria (=1) per poter scriverci. Per leggere da memoria esso deve essere 0.

2 Architettura

2.1 Schema del componente

Il componente è stato progettato utilizzando 3 contatori binari, 1 registro, 1 multiplexer e una macchina a stati finiti.

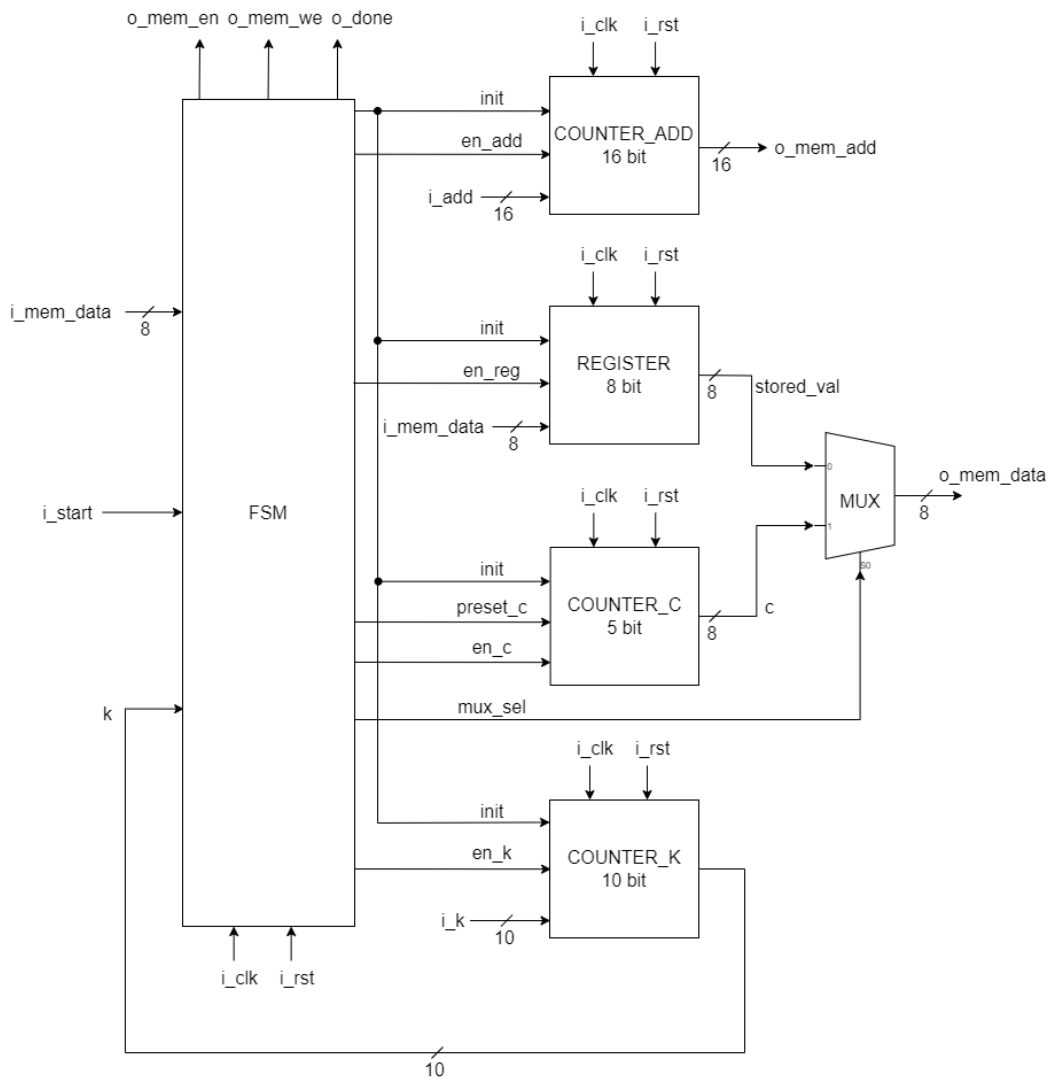


Figura 2: Schema del componente

2.2 Scelte progettuali

Il componente legge le parole dalla memoria e verifica che siano valori validi. In caso positivo memorizza il valore in un registro e scrive il valore di credibilità nella cella successiva. Altrimenti riempie la cella vuota con l'ultimo valore valido letto e scrive il valore di credibilità decrementato nella cella dopo. Per tenere conto dell'attuale indirizzo di memoria da elaborare e il numero di parole elaborate vengono utilizzati due contatori binari.

2.3 Moduli

Di seguito vengono descritti i vari moduli e la loro interfaccia, esclusi i segnali `i_clk` (clock globale che permette la gestione dei segnali sincroni) e `i_rst` (reset globale asincrono che riporta i valori memorizzati a zero), i quali sono presenti in ogni modulo.

2.3.1 counter_add

Contatore binario (upward) a 16 bit che contiene l'indirizzo di memoria che deve essere elaborato. I segnali (sincroni) del modulo sono:

- `i_load` è il segnale di load che permette il caricamento del valore iniziale
- `i_en` è il segnale di enable che abilita il contatore e incrementa di 1 il valore memorizzato
- `i_data` è il segnale (vettore) che contiene il valore che deve essere caricato nel contatore quando `i_load` è alto
- `output` è il segnale (vettore) di uscita che rappresenta il valore attuale del contatore

2.3.2 register

Registro binario a 8 bit che contiene l'ultimo valore diverso da zero letto dalla memoria. I segnali (sincroni) del modulo sono:

- `i_data` è il segnale (vettore) che contiene il dato da memorizzare
- `i_en` è il segnale di enable che abilita il registro e salva il valore presente sull'ingresso `i_data`
- `i_clr` è il segnale che resetta il componente riportando tutti i bit a 0
- `output` è il segnale (vettore) di uscita che rappresenta il valore memorizzato

2.3.3 counter_c

Contatore binario (downward) a 5 bit che contiene il valore di credibilità. I segnali (sincroni) del modulo sono:

- **i_preset** è il segnale di preset che reimposta il ciclo di conteggio portando il valore memorizzato a 31
- **i_en** è il segnale di enable che abilita il contatore decrementa di 1 il valore memorizzato nel contatore, solo se questo è maggiore di zero
- **i_clr** è il segnale che resetta il componente riportando tutti i bit a 0
- **output** è il segnale (vettore) di uscita che rappresenta il valore attuale del contatore (esteso su 8 bit)

2.3.4 counter_k

Contatore binario (downward) a 10 bit che decrementa il valore K per tenere il conto del numero di parole ancora da elaborare. I segnali (sincroni) del modulo sono:

- **i_load** è il segnale di load che permette il caricamento del valore iniziale
- **i_en** è il segnale di enable che abilita il contatore decrementa di 1 il valore memorizzato nel contatore
- **i_data** è il segnale (vettore) che contiene il valore che deve essere caricato nel contatore al quando **i_load** è alto
- **output** è il segnale (vettore) di uscita che rappresenta il valore attuale del contatore

2.3.5 MUX

Multiplexer che permette di selezionare quale valore deve essere scritto in memoria: il valore memorizzato in **register** se bisogna salvare l'ultimo valore letto, il valore memorizzato in **counter_c** se bisogna salvare il valore di credibilità.

2.4 Segnali

I seguenti segnali interni al componente gestiscono la comunicazione tra i vari moduli (vedi Figura 2).

- `init` è il segnale di inizializzazione per tutti i moduli (carica i valori iniziali di `counter_add` e `counter_k`, reimposta a 0 i valori memorizzati in `register` e `counter_c`);
- `preset_c` è il segnale che riporta il `counter_c` al valore di credibilità massimo;
- `en_add` è il segnale che abilita il conteggio del `counter_add`;
- `en_k` è il segnale che abilita il conteggio del `counter_k`;
- `en_c` è il segnale che abilita il conteggio del `counter_c`;
- `en_reg` è il segnale che abilita il `register` per memorizzare il valore letto dalla memoria;
- `stored_val` è il segnale che trasmette il valore memorizzato in `register` al multiplexer;
- `c` è il segnale che trasmette il valore memorizzato in `counter_c` al multiplexer;
- `mux_sel` è il segnale verso il multiplexer che seleziona l'ingresso corretto (0 per il valore di `register`, 1 per il valore di `counter_c`);
- `k` è il segnale che trasmette il valore di `counter_k` alla FSM.

2.5 FSM

La macchina a stati gestisce la logica del componente e i segnali che collegano i vari moduli per il loro corretto funzionamento.

È una macchina di Moore il cui schema è riportato in Figura 3.

Per una migliore leggibilità non sono state disegnate le transizioni che riportano la FSM nello stato iniziale quando viene alzato il reset globale.

2.5.1 Funzionamento

I due processi realizzano la funzione di transizione e la funzione di uscita. Il segnale `curr_state` memorizza lo stato attuale della macchina, che sarà uno dei seguenti. Tutte le transizioni tra stati avvengono al ciclo di clock successivo, dato che tutti i segnali sono sincroni.

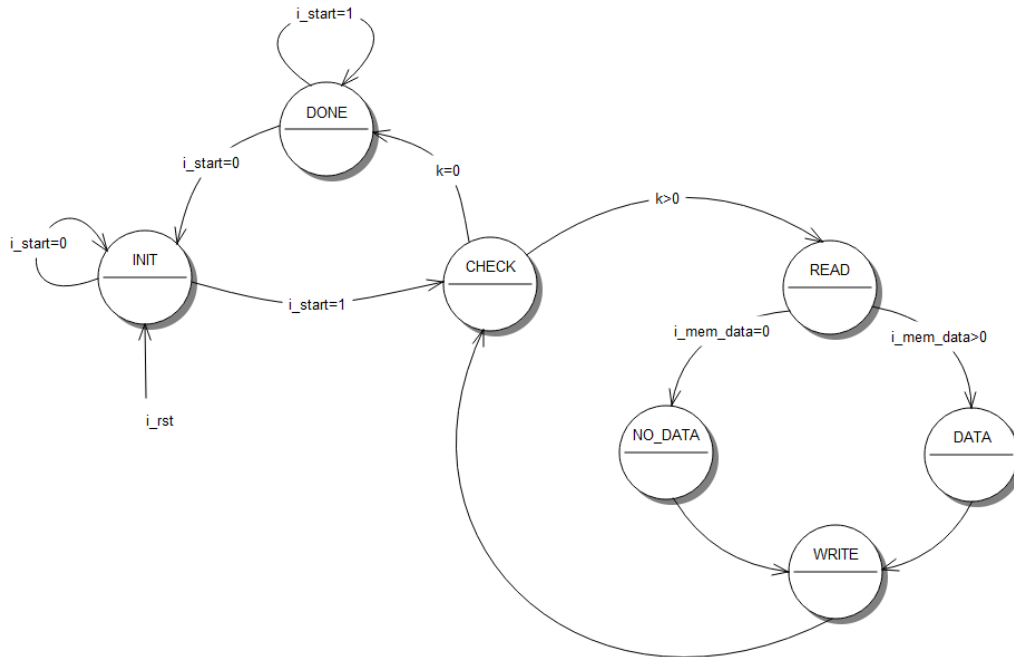


Figura 3: FSM

- (S0) **INIT** È lo stato iniziale a cui viene riportata la macchina quando viene alzato il segnale di reset globale. In questo stato vengono reinizializzati tutti i moduli tramite il segnale interno `init`. In particolare vengono caricati i valori presenti in ingresso `ADD` e `K` nei rispettivi contatori. Inoltre vengono resettati il registro e il contatore per la credibilità, portando i loro valori a zero. Quando `i_start` viene alzato la macchina passa allo stato **CHECK**.
- (S1) **CHECK** Questo stato verifica che il numero `k` proveniente dal contatore sia positivo. Questo significa che ci sono ancora parole da elaborare nella memoria e quindi si passa allo stato **READ**. Altrimenti, se `k` è uguale a zero significa che l'elaborazione è terminata e si passa allo stato **DONE**.
- (S2) **READ** In questo stato viene interpretata la parola proveniente dalla memoria sul segnale `i_mem_data`. Se rappresenta un numero positivo il valore letto è valido, quindi si passa allo stato **DATA**. Altrimenti, il numero zero è considerato come "valore non specificato" e quindi si passa allo stato **NO_DATA**.

- (S3) **DATA** Questo stato si occupa di memorizzare il valore proveniente dalla memoria nel registro, in quanto è l'ultimo valore valido letto. Inoltre deve reimpostare il contatore della credibilità al valore massimo (31). Infine incrementa il contatore di ADD per continuare l'elaborazione alla cella di memoria successiva e si passa allo stato **WRITE**.
- (S4) **WRITE** In questo stato viene scritto in memoria il valore di credibilità attuale contenuto nel rispettivo contatore. Bisogna quindi abilitare la memoria per la scrittura e selezionare il corretto input del multiplexer (1). Inoltre viene decrementato il valore di K contenuto nel contatore per segnalare che è stata terminata l'elaborazione di una parola. Infine viene incrementato il contatore ADD per elaborare la prossima cella di memoria. Poi si torna allo stato **CHECK**.
- (S5) **NO_DATA** Questo stato si occupa di riempire la cella memoria vuota con l'ultimo valore valido letto, che sarà contenuto nel registro. Per questo abilita la memoria in scrittura e seleziona l'input del multiplexer (0). Inoltre è necessario decrementare il valore di credibilità, dato che il valore nella cella attuale è "non specificato". Infine incrementa il contatore di ADD per continuare l'elaborazione nella cella successiva e si passa allo stato **WRITE**.
- (S6) **DONE** È lo stato finale che segnala il termine dell'elaborazione. Porta il segnale `o_done` a 1 e aspetta che il segnale `i_start` venga riportato a 0 per poter tornare allo stato **INIT** e attendere una nuova elaborazione.

La seguente tabella riassume i valori delle uscite per ogni stato.

	S0	S1	S2	S3	S4	S5	S6
<code>o_mem_en</code>	0	1	1	1	1	1	0
<code>o_mem_we</code>	0	0	0	0	1	1	0
<code>o_init</code>	1	0	0	0	0	0	0
<code>o_c_preset</code>	0	0	0	1	0	0	0
<code>o_add_en</code>	0	0	0	1	1	1	0
<code>o_k_en</code>	0	0	0	0	1	0	0
<code>o_c_en</code>	0	0	0	0	0	1	0
<code>o_reg_en</code>	0	0	0	1	0	0	0
<code>o_sel</code>	0	0	0	0	1	0	0
<code>o_done</code>	0	0	0	0	0	0	1

Tabella 3: Tabella delle uscite

3 Risultati sperimentali

Di seguito sono riportati i report post sintesi generati da Vivado.

3.1 Report Utilization

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	51	0	134600	0.04
LUT as Logic	51	0	134600	0.04
LUT as Memory	0	0	46200	0.00
Slice Registers	42	0	269200	0.02
Register as Flip Flop	42	0	269200	0.02
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

Il report mostra il numero di Flip Flop utilizzati per la sintesi del componente. In questo caso sono 42, infatti per ogni modulo sono necessari: 16 Flip Flop per il `counter_add`, 10 per il `counter_k`, 5 per il `counter_c`, 8 per il `register` e 3 per la codifica dei 7 stati della FSM. Inoltre si può notare che non è stato generato nessun Latch, coerentemente con le scelte di progetto.

3.2 Report Timing

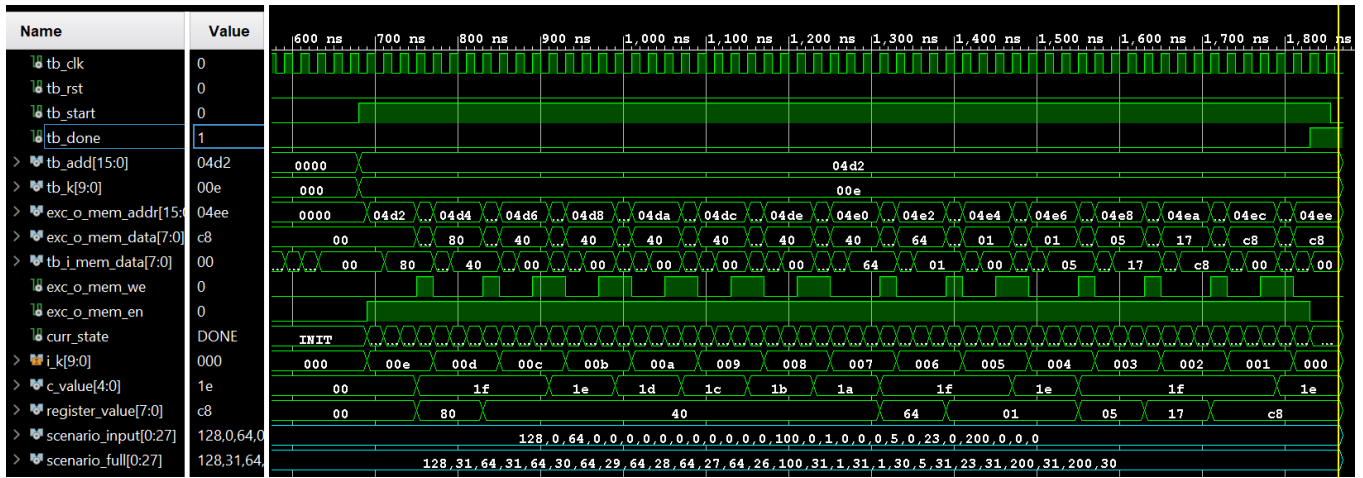
```
Slack (MET) :          16.947ns  (required time - arrival time)
Source:          counter_k/value_reg[4]/C
                 (rising edge-triggered cell FDCE clocked by clock  (rise@0.000ns fall@10.000ns period=20.000ns))
Destination:     counter_k/value_reg[9]/D
                 (rising edge-triggered cell FDCE clocked by clock  (rise@0.000ns fall@10.000ns period=20.000ns))
Path Group:      clock
Path Type:       Setup (Max at Slow Process Corner)
Requirement:     20.000ns  (clock rise@20.000ns - clock rise@0.000ns)
Data Path Delay:  2.902ns  (logic 0.993ns (34.218%)  route 1.909ns (65.782%))
Logic Levels:    3  (LUT4=1 LUT5=1 LUT6=1)
Clock Path Skew: -0.145ns  (DCD - SCD + CPR)
  Destination Clock Delay (DCD):  2.100ns = ( 22.100 - 20.000 )
  Source Clock Delay (SCD):  2.424ns
  Clock Pessimism Removal (CPR):  0.178ns
Clock Uncertainty: 0.035ns  ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
Total System Jitter (TSJ):  0.071ns
Total Input Jitter (TIJ):  0.000ns
Discrete Jitter (DJ):  0.000ns
Phase Error (PE):  0.000ns
```

Il report verifica che la rete soddisfa i requisiti di tempo. Infatti la differenza tra il tempo di clock (20 ns) e il tempo impiegato per produrre l'output è di 16.947 ns. Quindi il ritardo massimo della rete è 3.053 ns.

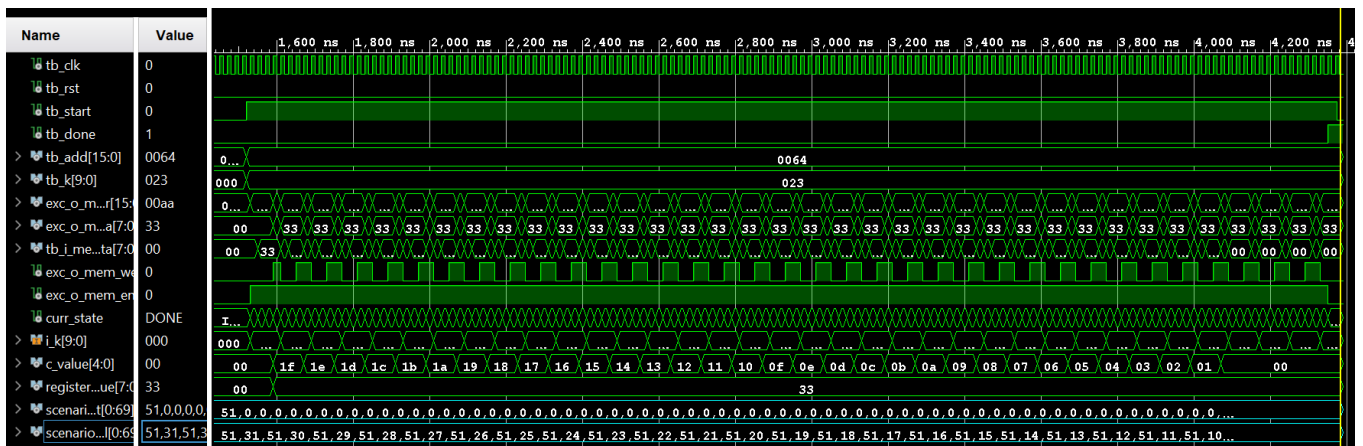
3.3 Testing

Si riportano i vari TestBench¹ sulla base dei quali il componente è stato testato per verificare che superasse i casi limite. Tutti i test sono stati superati con entrambe *Behavioral Simulation* e *Post-Synthesis Functional Simulation*.

Caso base TestBench di esempio fornito con la specifica: reset iniziale ed elaborazione di 14 parole di memoria partendo dall'indirizzo 0x04D2.

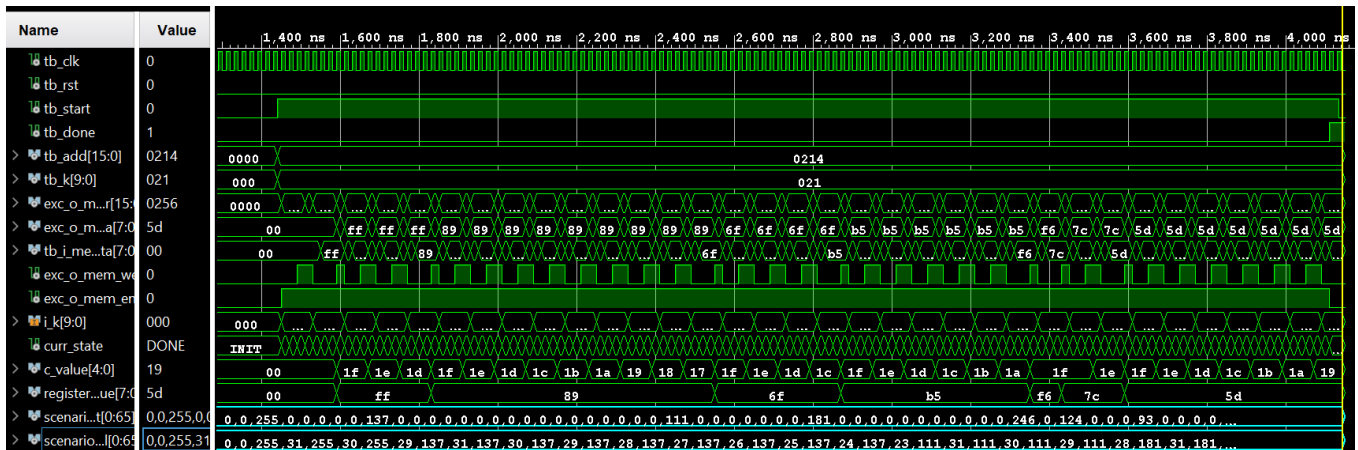


Caso particolare 1 *Esempio 2* della specifica: controlla che il valore di credibilità non scenda sotto zero. Come si può vedere dall'oscilloscopio, il c_value arriva a 0 dopo circa 4,000 ns, e poi non viene più decrementato.

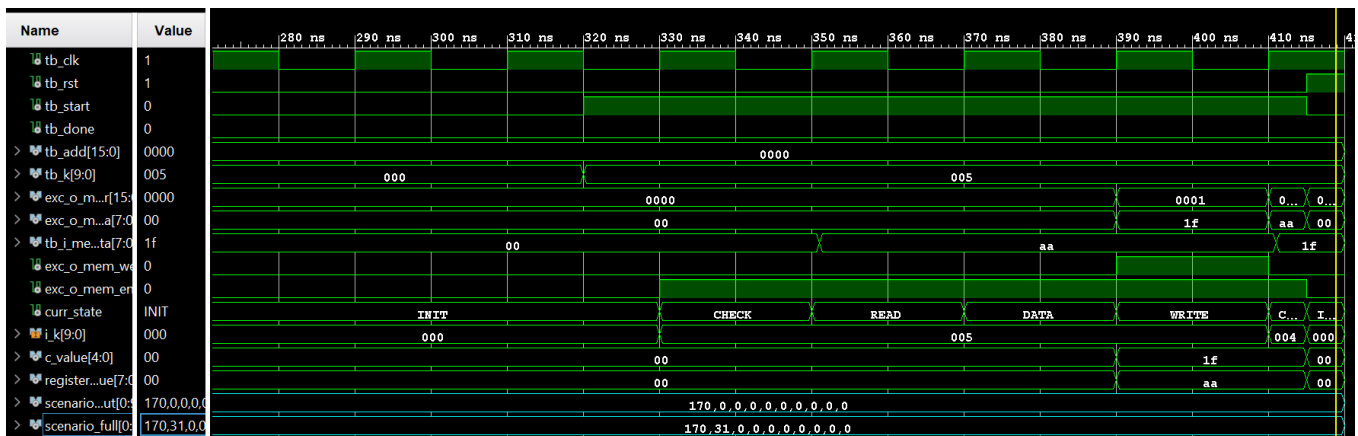


¹I segnali azzurri rappresentano gli scenari iniziali e finali delle elaborazioni.

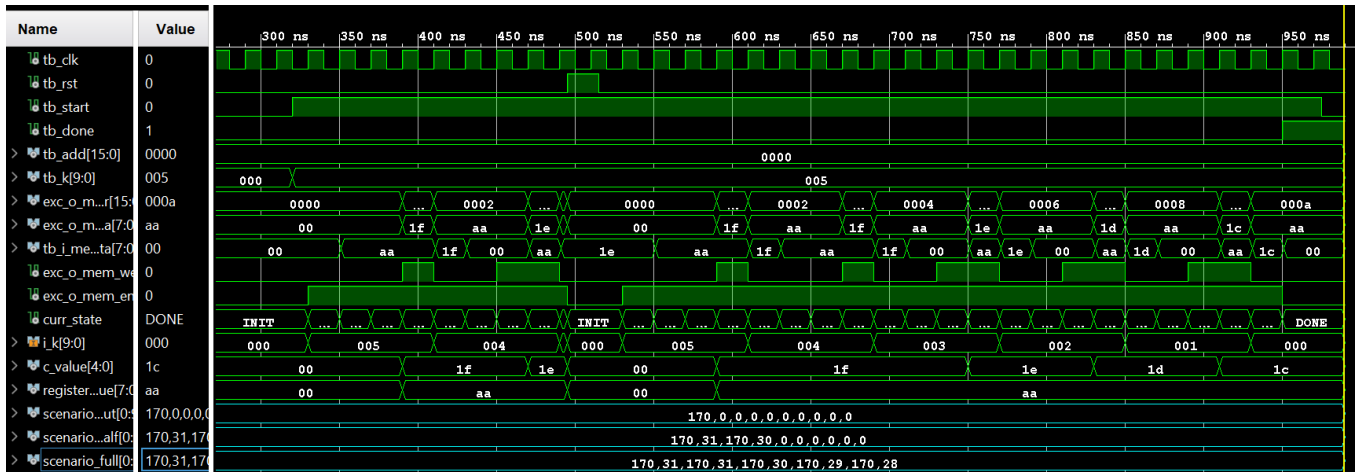
Caso particolare 2 *Esempio 3* della specifica: verifica il caso particolare dove la sequenza inizia con il valore zero. Il componente si comporta come da specifica, ovvero riscrive il valore 0 e lascia a 0 il valore di credibilità.



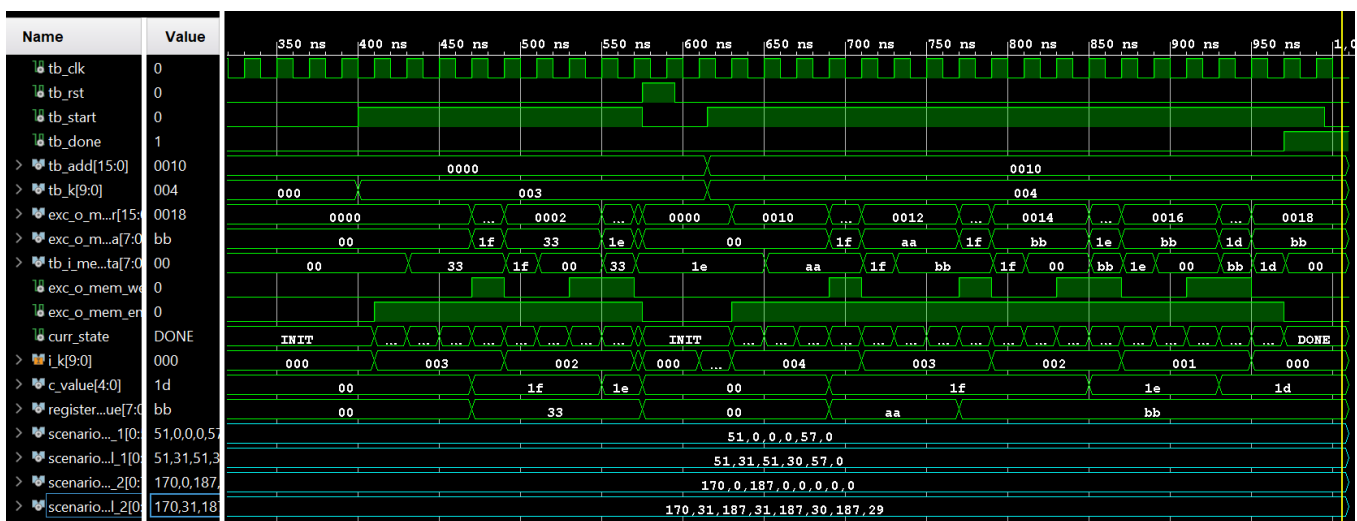
Reset Si considera un reset a metà elaborazione. In particolare il testbench aspetta la scrittura di un valore in memoria e poi alza il segnale di reset interrompendo l'elaborazione. Il componente si comporta come da specifica, riportandosi nello stato INIT.



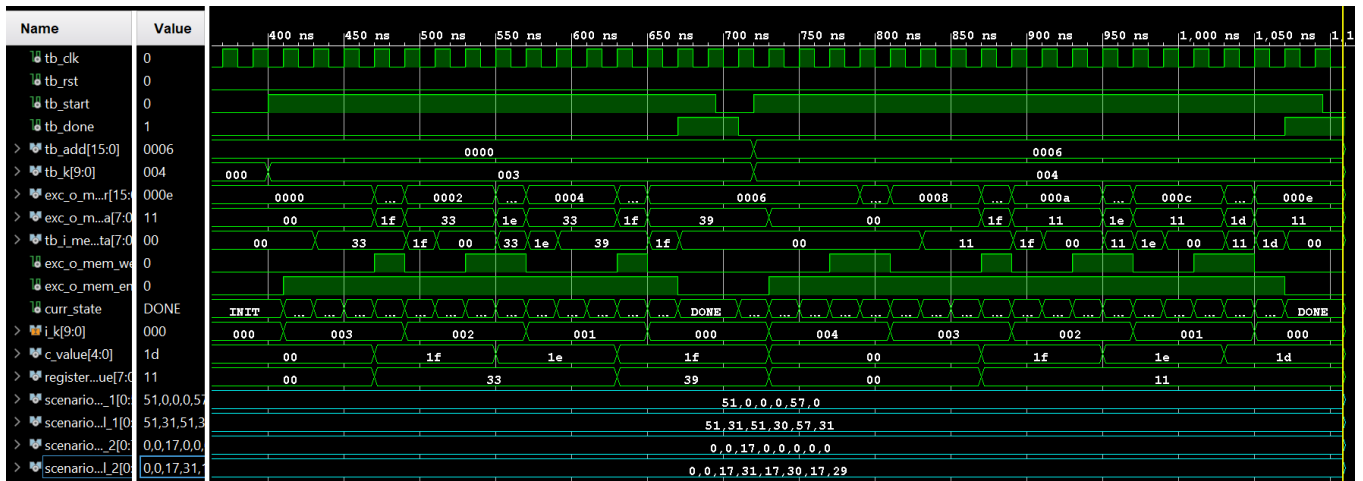
Reset + restart Si considera un reset dopo due scritture in memoria, poi l'elaborazione riparte da capo perché il segnale start non viene riportato a zero. In questo caso il componente si comporta in modo coerente. Fintanto che il segnale di reset è alto rimane nello stato INIT. Poi, al clock successivo, interpreta il segnale di start e ricomincia l'elaborazione.



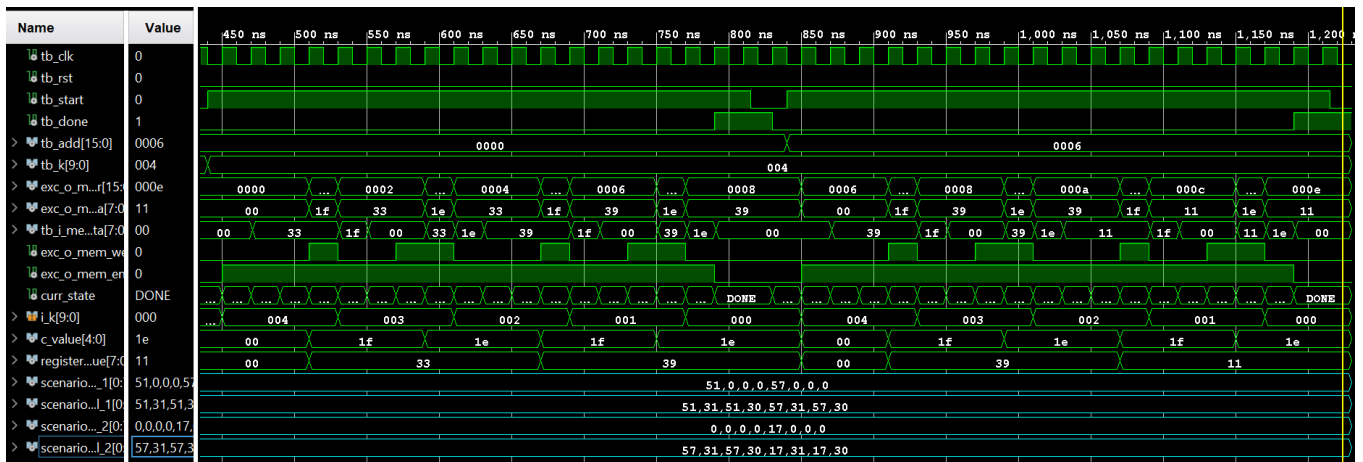
Reset + seconda elaborazione Si considera un reset dopo due scritture in memoria, in contemporanea il segnale start viene abbassato. Questa volta l'elaborazione rimane ferma anche dopo che il segnale di reset è stato abbassato, perché il segnale start non è più alto. Successivamente vengono forniti nuovi segnali ADD e K, start viene riportato a 1 e il componente riparte, lasciando incompleta l'elaborazione della prima sequenza.



Due elaborazioni successive Si considerano due elaborazioni complete consecutive. In particolare il secondo indirizzo di partenza è il successivo di quello finale della prima elaborazione. Come si può vedere dall'immagine, la seconda sequenza inizia con zero e il componente non ne modifica il valore: coerentemente con la specifica una seconda elaborazione non deve essere influenzata dalla prima.



Due elaborazioni sovrapposte Si considerano due elaborazioni che elaborano indirizzi di memoria sovrapposti. In particolare l'ultima parola della prima elaborazione (address 0x0006) viene considerata anche dalla seconda. Il secondo scenario non inizia con il valore 57 perché la memoria contiene zero. Poi però quell'indirizzo viene modificato e la seconda elaborazione si comporta di conseguenza: il valore è valido e la credibilità scritta è 31.



4 Conclusioni

Per realizzare il progetto, sono partito dall'analisi della specifica in modo da capire il funzionamento generale del componente. Poi ho individuato i moduli principali necessari per implementarlo. Successivamente sono passato alla realizzazione della FSM che gestisse le varie fasi dell'elaborazione. All'inizio avevo aggiunto uno stato in più per aspettare il primo valore proveniente dalla memoria. Poi però, analizzando la tabella delle uscite, ho realizzato che questo stato era equivalente allo stato CHECK.

La prima idea prevedeva che i segnali di load dei moduli fossero asincroni, perché pensavo così di poter ridurre il numero degli stati. Il componente funzionava correttamente, ma la sintesi generava dei Latch. Quindi li ho resi tutti sincroni, per una maggiore sicurezza implementativa, e ho notato che il numero degli stati della FSM non aumentava.

Prima di scrivere il codice VHDL ho provato a simulare un esempio di elaborazione manualmente, verificando che gli stati della FSM fossero corretti. Questo mi ha aiutato a verificare quanti e quali segnali erano necessari per sincronizzare tutti i componenti.

Infine, una volta realizzato il codice VHDL per i vari moduli, ho testato i diversi casi limite con successo, per essere sicuro che tutti i requisiti della specifica fossero soddisfatti.